EVOLUTIONARY COMPUTATION — EXERCISE 2

Greedy Regret Heuristics for the Node-Selection TSP

Greedy 2-regret · Weighted sum: 2-regret + best change of the objective function

Instances A and B · 200 nodes each · 100 selected per tour
Report generated 2026-10-19

Code at: github.com/mikolajdybizbanski/EC/blob/main/Lab2/main.cpp

Wiktor Bocheński · Mikołaj Dybizbański

1. Description of the problem

We are given two problem instances, **A** and **B**, each listing 200 nodes. Every node is described by three integers: its `x` and `y` coordinates on a plane, and a `cost`. Node indices are 0-based, taken from the row order in the instance file.

Goal. Select exactly 50% of the nodes (rounded up if the count is odd — for 200 nodes, exactly 100) and arrange the selected nodes into a **Hamiltonian cycle**: a closed path that visits every selected node exactly once and returns to its start. The objective to minimize is:

```
objective = total length of the cycle + sum of costs of the selected nodes
```

Distances between nodes are **Euclidean**, computed from the `(x, y)` coordinates and **rounded mathematically** to the nearest integer. The distance matrix is built once, immediately after reading an instance; from that point on every method only reads the distance matrix and the cost list.

A solution is a sequence of 100 distinct node indices, in visiting order; the cycle is implied by wrapping the last node back to the first.

Insertion cost. Every method in this report is driven by the change of the objective function caused by inserting an unselected node `c` between two consecutive cycle nodes `p` and `q`:

```
Δ(c, p, q) = distances[p][c] + distances[c][q] − distances[p][q] + costs[c]
```

2. Implemented methods

Both new methods extend the **greedy cycle** construction from Exercise 1: the solution is grown as a closed cycle, starting from two nodes. They change only *which* node is inserted at each step; the chosen node always goes to its best (cheapest) position. For every unselected node `c` we compute:

- **$\Delta_1(c)$** — the best (smallest) insertion cost of `c` over all edges of the current cycle,
- **$\Delta_2(c)$** — the second-best insertion cost,
- **2-regret** $r(c) = \Delta_2(c) - \Delta_1(c)$ — how much we lose if `c` is not inserted now and its best position is taken by another node.

1. **Greedy 2-regret** — insert the node with the **largest 2-regret**.
2. **Weighted sum (2-regret + best change of the objective)** — insert the node maximizing $score(c) = w \cdot r(c) - (1 - w) \cdot \Delta_1(c)$ with **equal weights, w = 0.5**. The objective change is a cost, so it enters the score with a minus sign.

Both methods are one function, `weightedRegretCycle(start, w)`: pure 2-regret is `w = 1`, the weighted sum is `w = 0.5` (and `w = 0` would reduce to the plain greedy cycle). Ties on the score are broken by the smaller Δ_1.

For each method, one solution is generated starting from **every** node (200 starts → 200 solutions per instance). The summary of results also includes the four methods from Exercise 1.

3. Pseudocode

3.1 Weighted 2-regret greedy cycle (common procedure)

```
weightedRegretCycle(distances, costs, start, w):
    second = argmin over c != start of ( distances[start][c] + costs[c] )
    tour = [start, second]
    used = {start, second}
    while |tour| < size:
        bestScore = -inf
        for each unvisited c:
            delta1 = +inf, delta2 = +inf
            for each cycle edge (tour[i], tour[i+1 mod |tour|]):
                delta = distances[tour[i]][c] + distances[c][tour[i+1]]
                        - distances[tour[i]][tour[i+1]] + costs[c]
                if delta < delta1:
                    delta2 = delta1
                    delta1 = delta, pos1 = i + 1
                else if delta < delta2:
                    delta2 = delta
            regret = delta2 - delta1
            score  = w * regret - (1 - w) * delta1
            if score > bestScore  (ties: smaller delta1 wins):
                bestScore = score
                (bestNode, bestPos) = (c, pos1)
        insert bestNode into tour at bestPos        // its best position
        mark bestNode as used
    return tour
```

3.2 Greedy 2-regret

```
greedy2Regret(distances, costs, start):
    return weightedRegretCycle(distances, costs, start, w = 1.0)
    // score(c) = regret(c) = delta2(c) - delta1(c)
```

3.3 Weighted sum: 2-regret + best change of the objective

```
weightedRegret(distances, costs, start):
    return weightedRegretCycle(distances, costs, start, w = 0.5)
    // score(c) = 0.5 * regret(c) - 0.5 * delta1(c)
```

Each step evaluates every unvisited node at every cycle edge, keeping only the two best insertion costs per node — the same O(n·k) work per step as the greedy cycle, so the regret variants cost practically the same.

4. Computational experiment

200 solutions per instance per method: 200 random restarts for the random method, and one run from each of the 200 starting nodes for every greedy method. Results of the previous methods (Exercise 1) are included for comparison; the new methods are highlighted. The best value in each column per instance is in bold.

Instance	Method	Min	Max	Average
A	Random solution	238710	294111	262519.0
A	Nearest neighbor (end only)	84362	90075	87476.6
A	Nearest neighbor (any position)	72328	77022	74347.0
A	Greedy cycle	72590	**75281**	73797.4
A	Greedy 2-regret	110303	128089	120368.0
A	Weighted 2-regret + best change	**72011**	75532	**73348.7**
B	Random solution	185377	231656	206675.0
B	Nearest neighbor (end only)	56007	**63166**	59397.3
B	Nearest neighbor (any position)	52915	66164	59480.3
B	Greedy cycle	53016	63736	59709.5
B	Greedy 2-regret	67078	76382	72368.0
B	Weighted 2-regret + best change	**52731**	63370	**57892.5**

Raw data: A_summary.csv, B_summary.csv.

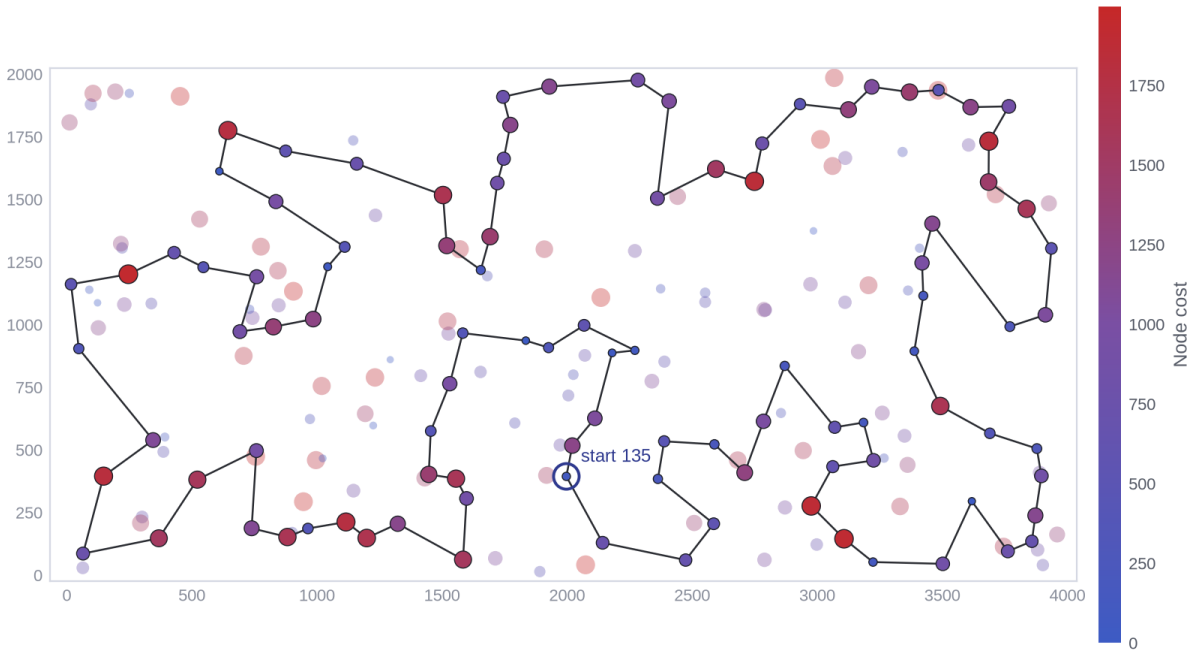
Running time of the new methods

Instance	Method	Time for 200 runs [s]
A	Greedy 2-regret	0.421
A	Weighted 2-regret + best change	0.442
B	Greedy 2-regret	0.498
B	Weighted 2-regret + best change	0.447

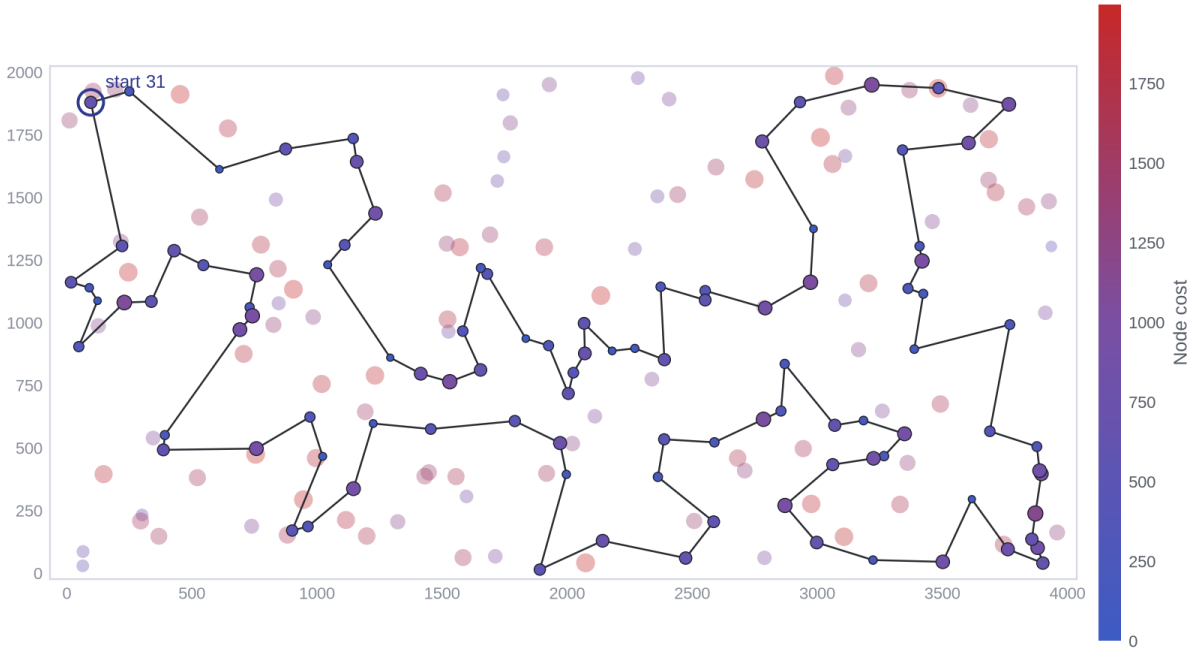
5. 2D visualizations of the best solution per instance/method

Node color and size encode cost (small/blue = cheap, large/red = expensive). Unselected nodes use the same color and size scale but are drawn faint and without an outline; the closed tour is drawn as a dark path; the ring marks the start node of the best run.

Instance A

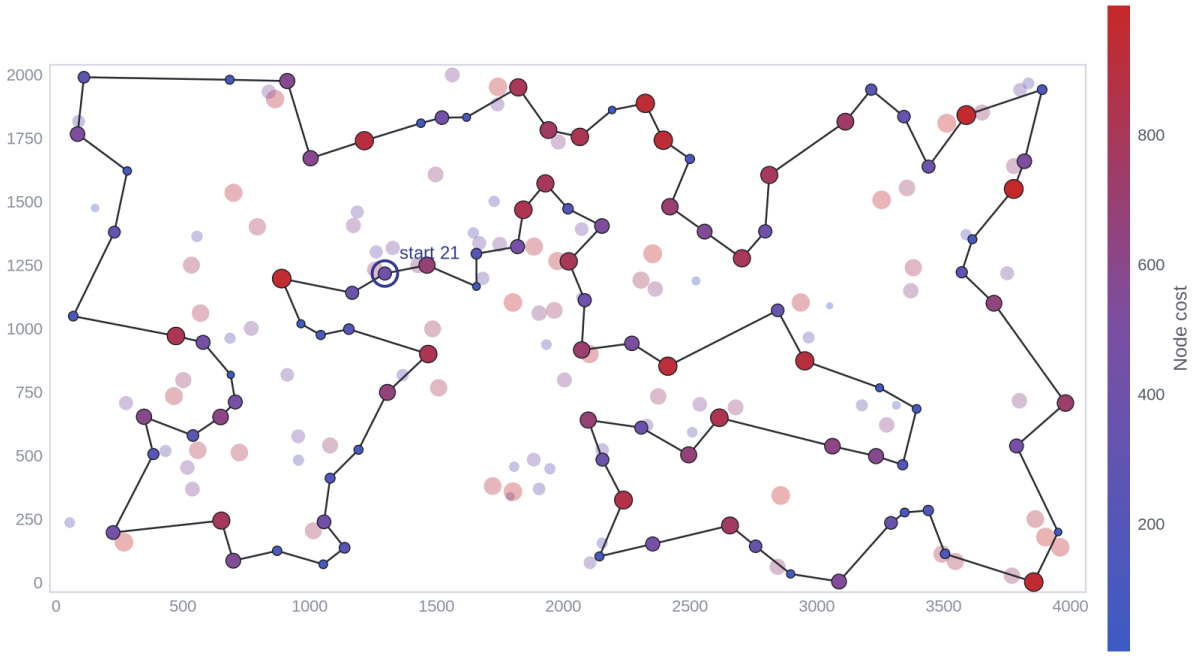


Greedy 2-regret · objective 110303

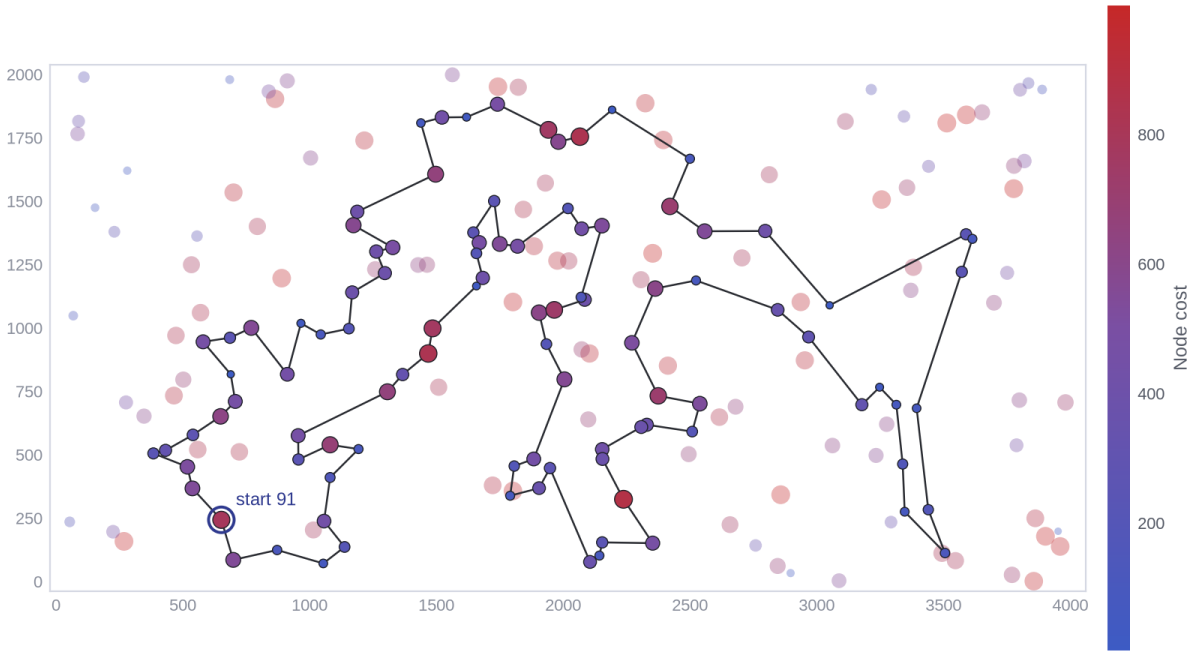


Weighted 2-regret + best change · objective 72011

Instance B



Greedy 2-regret · objective 67078



Weighted 2-regret + best change · objective 52731

6. Best solutions (node indices, 0-based)

Instance A, greedy 2-regret `objective 110303`

```
135, 181, 192, 158, 58, 84, 134, 65, 72, 174, 150, 106, 42, 151, 165, 105, 57, 64, 107, 88, 153, 75, 170,
159, 130, 117, 93, 50, 169, 59, 47, 37, 147, 6, 46, 86, 55, 17, 29, 19, 163, 24, 136, 186, 109, 34, 62,
154, 39, 175, 115, 5, 112, 139, 133, 51, 196, 18, 71, 91, 137, 80, 38, 76, 132, 162, 148, 160, 100, 152,
108, 167, 116, 168, 95, 188, 129, 110, 120, 161, 92, 32, 2, 166, 183, 10, 22, 98, 70, 27, 87, 102, 35, 67,
144, 52, 36, 177, 41, 61
```

Instance A, weighted 2-regret + best change `objective 72011`

```
31, 60, 116, 149, 63, 168, 11, 49, 108, 152, 100, 123, 83, 160, 97, 127, 161, 30, 187, 171, 2, 189, 141,
87, 45, 73, 135, 114, 181, 192, 158, 58, 84, 134, 72, 176, 174, 150, 106, 113, 89, 42, 151, 138, 182, 57,
64, 107, 88, 78, 193, 153, 75, 170, 146, 159, 130, 47, 93, 50, 3, 169, 1, 178, 101, 55, 29, 163, 136, 186,
164, 142, 8, 9, 54, 33, 131, 36, 177, 52, 179, 53, 140, 144, 67, 16, 51, 35, 185, 102, 28, 180, 132, 76, 4,
71, 157, 91, 80, 48
```

Instance B, greedy 2-regret `objective 67078`

```
21, 38, 161, 154, 140, 47, 4, 33, 40, 60, 157, 45, 134, 175, 62, 91, 174, 132, 103, 6, 180, 115, 170, 177,
53, 65, 145, 149, 13, 37, 116, 42, 50, 195, 121, 198, 96, 164, 43, 32, 89, 44, 83, 57, 79, 185, 144, 104,
77, 194, 190, 70, 187, 165, 105, 61, 193, 114, 10, 143, 131, 118, 110, 23, 2, 14, 20, 74, 17, 171, 99, 163,
88, 87, 126, 181, 25, 34, 31, 119, 150, 182, 153, 90, 127, 106, 162, 35, 27, 111, 3, 137, 86, 72, 166, 92,
117, 172, 69, 168
```

Instance B, weighted 2-regret + best change `objective 52731`

```
91, 36, 197, 132, 29, 6, 180, 115, 170, 177, 125, 26, 97, 154, 140, 47, 38, 21, 30, 183, 191, 130, 63, 121,
198, 96, 48, 43, 156, 32, 89, 57, 79, 185, 104, 54, 196, 114, 10, 90, 14, 2, 20, 153, 129, 127, 80, 192,
162, 76, 135, 27, 11, 66, 67, 24, 34, 152, 181, 126, 88, 179, 87, 82, 167, 51, 85, 59, 73, 98, 151, 122,
173, 3, 184, 86, 46, 72, 117, 8, 120, 133, 136, 172, 160, 69, 124, 4, 64, 33, 155, 71, 141, 40, 60, 157,
45, 134, 175, 62
```

7. Solution checker

All 4 best solutions above were checked with `checker.py`, which independently re-reads the instance data, counts the visited nodes and recomputes the objective (cycle length + node costs) from scratch. We additionally verified that no index repeats.

Instance	Method	Nodes	main.cpp	checker.py
A	Greedy 2-regret	100	110303	110303
A	Weighted 2-regret + best change	100	72011	72011
B	Greedy 2-regret	100	67078	67078
B	Weighted 2-regret + best change	100	52731	52731

ALL 4 VALID — the recomputed objective matched the value reported by `main.cpp` exactly in every case.

8. Conclusions

- **Pure greedy 2-regret performs poorly** — it is the worst of all greedy methods on both instances (average 120368 on A and 72368 on B, vs. 87477 / 59397 for the simplest nearest neighbor). The reason is structural: the node cost appears in both Δ_1 and Δ_2, so it *cancels out* of the regret. The method is blind to node costs and to how far a node is from the cycle, and it happily pulls in distant, expensive nodes whose regret is high only because they are far away. The plots confirm this — the w = 1 tours contain many large red (expensive) nodes. In this problem the key decision is *which* nodes to visit, and regret alone does not address it.
- **The weighted sum (0.5 · regret − 0.5 · best change) is the best method so far.** On A it found the best solution overall (72011, vs. 72328 for NN-any-position) and the best average (73348.7, vs. 73797.4 for greedy cycle). On B it is best in all three columns — min 52731, max 63370, average 57892.5, about 1500 lower on average than any Exercise 1 method.
- The two terms complement each other: the objective-change term keeps every insertion cheap, while the regret term prevents the typical greedy mistake of postponing a node that has only one good slot until that slot is gone.
- The weighted method is also **robust to the start node**: on B its worst run (63370) is better than the worst runs of NN-any-position (66164) and greedy cycle (63736); on A its min–max spread is similar to greedy cycle.
- **Cost:** the regret variants evaluate the same insertions as the greedy cycle and only keep the second-best value as well, so the running time is practically unchanged (≈0.42–0.50 s for 200 runs).
- Overall, regret is useful only *together with* the objective change; with equal weights it already improves on every constructive heuristic from Exercise 1.